Scale-Aware Evolutionary Discovery of Wind-Farm Layout Optimizers with Large Language Models

Author One Author Two Author Three
ADDRESS
Correspondence: (EMAIL)

Abstract

We present a framework for the automated discovery of *deployment-candidate* learning-rate and constraint-penalty schedules for gradient-based wind-farm layout optimization using large language models (LLMs). Candidate schedules are expressed in a *scale-aware* parameterization in which the exploration learning rate is constructed from the rotor diameter and the only user-supplied scale is the constraint tolerance, so a single schedule is scale-covariant and can be applied across farms of different size with no free per-farm parameter. An island-based, quality-diversity evolutionary loop drives LLM coding agents to propose schedules, which a multi-stage *cascade* evaluator scores across a multi-farm, multi-scale training suite under a hard feasibility gate. A behavioral archive, content-addressed caching, and complete lineage provenance make every discovery auditable and, on a fixed platform, bit-reproducible; a firewall discipline keeps held-out and test performance out of the search. We describe the schedule interface, the fixed optimization skeleton, the evolutionary loop and cascade evaluator, the fitness, selection, and deployment rules, and the reproducibility and firewall machinery that together turn opportunistic program search into a repeatable engineering procedure. Empirically the search is near-null in mean AEP — discovered schedules beat the native scale-aware reference by only $\approx$0.05–0.09% on the training cells. At the deployment-relevant metric, the best of a converged multistart, a discovered schedule beats native by a comparable but *statistically robust* +0.075% (bootstrap 95% confidence interval excluding zero) on a held-out farm firewalled from the search; the value added is robustness at the deployed metric, not a larger effect. The gain is small and farm-specific rather than uniform, is computed on our reimplemented skeleton, and rests on a schedule selected as a search argmax, so we report it as a deployment *candidate* and define a pre-registered test to confirm or refute it.

1 Introduction

When designing a wind farm, the placement of turbines within the buildable area determines every downstream stage of a project, from land valuation to annual energy production (AEP). Because a fraction-of-a-percent gain in modeled AEP can translate into a large change in a project's financial case, layout optimization is a well-studied problem: it is a non-convex, non-linear, constrained optimization in which turbine positions interact through wake losses and must respect spacing and boundary (inclusion/exclusion zone) constraints.

Gradient-based solvers have become a de-facto method for this problem. The TopFarm stochastic gradient descent (SGD) optimizer (Quick et al., 2022) adapts the Adam algorithm (Kingma and Ba, 2015) with a scheduled learning rate and a scheduled linear constraint penalty. The *schedule* — how the learning rate, the penalty weight, and the two Adam moment-decay coefficients evolve over the course of the optimization — is a rich design space that governs the trade-off between exploring the layout and driving the final iterate onto the feasible set. It is precisely the kind of low-dimensional, high-leverage function that is difficult to design by hand and attractive to search automatically.

Large language models have recently been used as the mutation operator in evolutionary *program search*: FunSearch (Romera-Paredes et al., 2024) and AlphaEvolve (Google DeepMind, 2025) evolve populations of programs by repeatedly prompting an LLM to improve high-scoring candidates. We adopt this paradigm for optimizer-schedule design, and add three ingredients aimed at making the discovered artifact a *deployment candidate* rather than merely high-scoring on one farm:

1. a **scale-aware schedule interface** (Section 3.2) that removes the free learning-rate parameter and builds the exploration scale from the rotor diameter, so a discovered schedule is scale-covariant and transfers across farms of different turbine size;
2. a **quality-diversity evolutionary loop with a cascade evaluator** (Sections 3.5–3.7) that scores candidates across a multi-farm, multi-scale training suite under a hard feasibility gate, rejecting infeasible or dominated schedules cheaply before spending on expensive high-count evaluations; and
3. a **provenance, firewall, and reproducibility layer** (Sections 3.9–7) — full lineage, a behavioral archive, content-addressed caching, a scoped mutator workspace, and a pre-registered selection and deployment criterion — so that any "the system discovered X" claim is auditable against what was seeded and what was measured.

The remainder of the paper positions the approach (Section 2), describes the framework (Section 3), the wind farms and evaluation cells it is applied to (Section 4), the schedules the search discovers and their held-out deployment (Sections 5–6), and the reproducibility, limitations, and cost-control machinery (Sections 7, 8).

2 Related work

LLM-driven program and algorithm search. FunSearch (Romera-Paredes et al., 2024) and AlphaEvolve (Google DeepMind, 2025) use an LLM as the mutation operator in an evolutionary loop over programs scored by a fixed evaluator; ShinkaEvolve (Sakana AI, 2025) and OpenEvolve (OpenEvolve, 2025) add sample efficiency and open-source island/MAP-Elites scaffolding. We adopt this paradigm but target an *optimizer schedule* — a small, scale-covariant control function — rather than a free-form program, and add the deployment discipline (a scale-aware interface, a hard feasibility gate, held-out out-of-sample evaluation, an information firewall, and provenance) that a schedule intended for field use requires.

LLMs for optimization and hyperparameters. A parallel line uses LLMs to propose optimizer or hyperparameter settings directly: OPRO (Yang et al., 2023) treats the LLM as an optimizer that iteratively proposes solutions from a natural-language trajectory, and Eureka (Ma et al., 2023) has an LLM write and refine reward functions for control. Our target is adjacent but distinct: a deployable, scale-covariant *schedule function* for a specific constrained engineering optimizer, selected and tested behind a firewall rather than optimized in the loop against the target metric.

Wind-farm layout optimization. Gradient-based layout optimization with the TopFarm SGD driver (Quick et al., 2022) and inclusion/exclusion-zone handling (Criado Risco et al., 2024), over an engineering wake model (Bastankhah and Porté-Agel, 2014), is the substrate the schedule controls; the multistart-convergence protocol we use to size the deployment baseline follows Quick et al. (2026). To our knowledge this is the first work to *discover* the learning-rate and constraint-penalty schedule for this class of solver with an LLM and to evaluate the discovery as a deployment candidate across farms of different scale.

3 Framework

This section describes the discovery framework: the optimization problem (3.1), the scale-aware schedule interface (3.2), the fixed optimization skeleton (3.3), the penalty normalization (3.4), the evolutionary loop (3.5) and its behavioral archive (3.6), the cascade evaluator (3.7), the fitness and feasibility rules (3.8), and the provenance, firewall, selection, and deployment machinery (3.9–3.11). Figure 1 gives the overall data flow.

3.1 Optimization problem

For a farm with N turbines at horizontal positions $(\boldsymbol{x}, \boldsymbol{y})$ we maximize the modeled annual energy production

$$\max_{\boldsymbol{x},\boldsymbol{y}} \; J(\boldsymbol{x}, \boldsymbol{y}), \qquad J = \frac{8760}{10^6} \sum_k w_k \, P\Big(\boldsymbol{x}, \boldsymbol{y};\, u_\infty^{(k)}, \theta^{(k)}\Big), \tag{1}$$

where P is the wind-farm power under inflow speed $u_\infty^{(k)}$ and direction $\theta^{(k)}$ with probability weight w_k, evaluated with an engineering wake model (Bastankhah and Porté-Agel, 2014), subject to the feasibility constraints

$$d(x_i, y_i) \geq 0 \;\; \forall i, \qquad \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \;\geq\; D_{\min} \;\; \forall i \neq j, \tag{2}$$

where $d(x_i, y_i)$ is the signed distance from turbine i to the nearest inclusion-zone boundary (negative outside the buildable region) and $D_{\min}$ is the minimum inter-turbine spacing. Feasibility is judged against a tolerance $\gamma_{\min}$ (in metres): a layout is feasible when every turbine satisfies the boundary constraint to within $\gamma_{\min}$ and the pairwise spacing constraint. $\gamma_{\min}$ is the single user-supplied scale of the problem.

3.2 Scale-aware schedule interface

The object we search over is a pure function that returns, at each optimization step i, the four quantities the skeleton needs: a learning rate η_i, a constraint penalty multiplier α_i, and the two Adam moment-decay coefficients $\beta_{1,i}, \beta_{2,i}$:

$$\big(\eta_i, \alpha_i, \beta_{1,i}, \beta_{2,i}\big) = \mathcal{S}\big(i,\, T,\, D,\, D_{\min},\, N,\, \gamma_{\min},\, \alpha_0\big). \tag{3}$$

The arguments are the current step i, the total number of steps T, and the *problem-intrinsic scales*: the rotor diameter D, the minimum spacing $D_{\min}$, the turbine count N, the metre-valued constraint tolerance $\gamma_{\min}$, and a skeleton-computed penalty normalizer α_0 (Section 3.4). The Python signature is reproduced in Appendix A.

Two design choices make a discovered schedule deployable across scales:

No free learning rate. There is no driver-supplied initial learning rate η_0 anywhere in Equation (3). The exploration scale must be constructed *inside* the schedule from the rotor diameter, e.g. a peak learning rate $\eta_{\text{peak}} = c_0 \, D$ with c_0 a schedule-internal constant or curve that the search may evolve but that is never a user knob. Because the turbine spacing, wake length, and feasible-region geometry all scale with D, expressing the exploration scale as a multiple of D makes the schedule scale-covariant: the same function applied to a farm with a different rotor diameter automatically takes appropriately sized steps.

Tolerance as the terminal learning rate. Following TopFarm's own convention, the terminal learning rate is a proxy for how tightly the boundary is ultimately satisfied; we expose it directly as the metre-valued tolerance $\gamma_{\min}$ toward which the schedule decays the learning rate. It is the only user-supplied scale, and a faithful schedule must respond to it: at a looser $\gamma_{\min}$ the schedule should stop decaying sooner, and its terminal behavior and feasibility should change accordingly (this responsiveness is checked in Section 3.7).

Algorithm 1 Fixed optimization skeleton (the search controls only $\mathcal{S}$)

$\boldsymbol{s} \leftarrow$ wind-aware init$(D, D_{\min}, N, \text{resource})$; $\quad \boldsymbol{m}, \boldsymbol{v} \leftarrow \boldsymbol{0}$
$\alpha_0 \leftarrow \text{mean}\big(|\nabla_{\boldsymbol{s}} J(\boldsymbol{s})|\big)/D$; $\quad \alpha_0 \leftarrow \textsc{canonicalize}(\alpha_0)$ ▷ Section 3.4
for $i = 0, 1, \ldots, T-1$ **do**
  $(\eta_i, \alpha_i, \beta_{1,i}, \beta_{2,i}) \leftarrow \mathcal{S}(i, T, D, D_{\min}, N, \gamma_{\min}, \alpha_0)$
  $\boldsymbol{j} \leftarrow \nabla_{\boldsymbol{s}}\big[-J(\boldsymbol{s})\big] + \alpha_i \nabla_{\boldsymbol{s}} \Phi(\boldsymbol{s})$ ▷ objective + penalty gradient
  $\boldsymbol{m} \leftarrow \beta_{1,i}\boldsymbol{m} + (1-\beta_{1,i})\boldsymbol{j}$; $\quad \boldsymbol{v} \leftarrow \beta_{2,i}\boldsymbol{v} + (1-\beta_{2,i})\boldsymbol{j}^2$
  $\hat{\boldsymbol{m}} \leftarrow \boldsymbol{m}/(1-\beta_{1,i}^{i+1})$; $\quad \hat{\boldsymbol{v}} \leftarrow \boldsymbol{v}/(1-\beta_{2,i}^{i+1})$
  $\boldsymbol{s} \leftarrow \boldsymbol{s} - \eta_i\, \hat{\boldsymbol{m}}/\sqrt{\hat{\boldsymbol{v}}}$
end for
return $\boldsymbol{s}$

3.3 Fixed optimization skeleton

The schedule is the *only* thing the search controls. A fixed skeleton (Algorithm 1) handles initialization, gradient computation, and the Adam update. It (i) places the initial layout on a wind-direction-aware grid inside the buildable region (for the multi-zone case, on a zone-interior fill); (ii) forms the objective gradient of Equation (1) and the constraint gradient of the boundary and spacing penalties by automatic differentiation; (iii) combines them as $\boldsymbol{j} = \nabla_{\text{obj}} + \alpha_i \nabla_{\text{con}}$; and (iv) applies an Adam step with the scheduled $\eta_i, \beta_{1,i}, \beta_{2,i}$. The horizon is fixed at $T = 8000$ steps. Nothing in the skeleton contains a hardcoded learning rate; the entire step-size behavior comes from Equation (3).

The penalty is the sum of a boundary and a spacing term, both quadratic and scale-matched,

$$\Phi(\boldsymbol{x}, \boldsymbol{y}) \;=\; \sum_i \min\big(0,\; d(x_i, y_i)\big)^2 \;+\; \sum_{i<j} \frac{\max\big(0,\; D_{\min}^2 - r_{ij}^2\big)^2}{D_{\min}^2}, \tag{4}$$

with r_{ij} the pairwise turbine distance. We use a quadratic (squared-violation) spacing penalty normalized by $D_{\min}^2$ rather than the linear/exact-penalty form of the underlying solver: the quadratic form is smooth and keeps the boundary and spacing terms on a common metre^2 scale, so a single schedule-controlled multiplier α_i balances both across farms of different rotor diameter. We treat this as a modeling choice motivated by scale-matching rather than a measured improvement; we do not ablate the linear and quadratic forms here. Feasibility itself (Equation (2)) is judged geometrically at tolerance $\gamma_{\min}$, independently of the penalty that drives the optimization toward it.

3.4 Penalty normalization

So that the penalty multiplier is on a problem-intrinsic scale, the skeleton computes the normalizer

$$\alpha_0 = \frac{\text{mean}\,\big(|\nabla_{\boldsymbol{x}} J|,\; |\nabla_{\boldsymbol{y}} J|\big)}{D} \tag{5}$$

at the initial layout and passes it into Equation (3); the schedule composes the running penalty α_i from α_0. Normalizing the gradient magnitude by the rotor diameter (rather than by a learning rate) keeps the penalty scale covariant with the geometry and independent of the step-size choice, consistent with the removal of a free learning rate. To make evaluation bit-reproducible across *independent processes on a given platform*, α_0 is canonicalized to single precision at the skeleton boundary (CANONICALIZE in Algorithm 1): the double-precision gradient reduction agrees to far more significant figures than single precision, so rounding α_0 to a canonical single-precision value makes the same (schedule, cell, seed) reproduce identically in independent processes. This matters because the low–learning-rate tail of the optimization is sensitive to α_0. Across *different* hardware architectures a small floating-point drift remains; we therefore report cross-platform agreement to within a measured drift tolerance (Section 7) rather than bit-identity.

Algorithm 2 Evolutionary schedule discovery

seed each island archive with generation-0 ancestors (logged with lineage)
while cost < ceiling **do**
  for each island **do**
    $p \leftarrow$ SAMPLEPARENT(archive) ▷ fitness/novelty-aware
    materialize a scoped mutator workspace (Section 3.10)
    $c \leftarrow$ LLM-MUTATE(p, firewall-safe feedback)
    if NOVELTYFILTER rejects c **then continue**
    end if
    (fit, feasible, desc) $\leftarrow$ CASCADE(c) ▷ Section 3.7
    if feasible **then** ARCHIVEINSERT(island, c, fit, desc)
    end if
    LOGLINEAGE(c, p, engine, tokens, cost, desc, fit)
  end for
  periodically migrate elites between islands; checkpoint state
end while

3.5 Evolutionary discovery loop

Schedules are discovered by a quality-diversity evolutionary loop (Algorithm 2), built on an island model with a MAP-Elites archive (Mouret and Clune, 2015) per island. Each iteration: (i) a parent schedule is sampled from an island's archive with a fitness- and novelty-aware sampler; (ii) an LLM mutation engine is prompted with the parent source and firewall-safe feedback and returns an improved schedule as Python source; (iii) a code-novelty filter rejects a child that is a near-duplicate of an already-seen program before any expensive evaluation is spent; (iv) the survivor is scored by the cascade evaluator (Section 3.7); and (v) it is inserted into the island's behavioral archive if it is elite for its cell. Islands periodically exchange elites. The loop runs until a token/cost ceiling is reached (Section 7).

The population is *seeded* with known-good schedules, logged as generation-0 ancestors with their port transforms (Section 3.9); the search is an engineering procedure and seeding is encouraged. The mutation operators are LLM coding agents accessed through a subscription/OAuth path: the Claude Agent SDK, a command-line Gemini engine, and a command-line OpenAI ("codex") engine, with a deterministic mock engine for testing. The two searches shown below (Figure 4) use the Claude and Gemini engines; the deployed schedule comes from a separate codex-engine portfolio run, not plotted (Section 5). We adopt two eval-saving mechanisms from ShinkaEvolve (Sakana AI, 2025): code-novelty rejection-sampling and fitness/novelty-aware parent sampling; the island/MAP-Elites and cascade scaffolding follow OpenEvolve (OpenEvolve, 2025).

3.6 Behavioral archive

Each candidate is placed in a MAP-Elites archive keyed by four behavioral descriptors computed from the schedule function *before* evaluation, so that the archive covers qualitatively distinct schedule shapes rather than crowding a single family (Table 1): the peak learning rate relative to the rotor diameter, the terminal learning rate in metres, the coupling between the penalty and the learning rate, and the number of learning-rate restarts. Within each island, one elite is kept per occupied cell; feasibility dominates, then mean fitness, then a worst-cell tiebreak (Section 3.8). Seeding populates several descriptor cells from generation 0.

Table 1: Behavioral descriptors and their (frozen) bins for the MAP-Elites archive. Descriptors are computed from the schedule function prior to evaluation.

Descriptor	Bins
Peak learning rate / D	< 0.5, 0.5–0.8, 0.8–1.2, >1.2
Terminal learning rate (m)	≤ 0.01, 0.01–0.1, 0.1–1, >1
Penalty coupling	coupled, decoupled, cyclic
Learning-rate restarts	0, 1–2, ≥ 3

3.7 Cascade evaluator

Evaluating a schedule on every training cell at full seed count is expensive, so candidates pass through a cascade that spends compute in proportion to promise. Each stage runs the fixed skeleton (Algorithm 1) on a cell/seed and records the resulting AEP and feasibility.

Stage A (gross fast-reject). Two cheap cells × two seeds. This is a deliberately *gross* filter: a candidate is rejected only if any seed is infeasible or its AEP is more than about one percent below that cell's reference. It is *not* texture-floor-tight — a floor-tight Stage A would mass-reject the very exploration the quality-diversity archive exists to keep, so the texture floors are reserved for selection margins, not for this filter. The rejection rate by cause (infeasible vs. below-reference) is logged as a pilot diagnostic. No high-count cell is used here.

Stage B (training matrix). The full frozen training set (Table 2) × at least five *paired* seeds — the same seed produces the same wind-aware initial layout for the candidate and for the reference, so per-seed differences are paired. Each cell yields a percentage score (Section 3.8) and a candidate-feasibility flag; the cross-cell aggregate is the candidate's stage-B fitness.

Stage B$^+$ (elite tier). Top-k archive elites are re-scored on expensive high-count cells, where a longer per-evaluation budget is affordable. This stage runs only on dedicated compute and never inside the main loop. It also carries the *capability-frontier* cells — cells (a high-count single-polygon farm and a dense multi-zone farm under a unidirectional resource) on which even the reference schedule is infeasible. These are tracked at the elite tier as qualitative probes: a candidate that reaches strict feasibility there is a qualitatively new result. They never gate the per-generation search; off the dedicated compute they are simply marked pending and deferred.

Stage C (holdout & responsiveness). Elites are scored on the held-out farm as an *out-of-sample generalization check* of the already-selected schedule (selection maximizes farm-balanced *training* fitness; the held-out farm is not a selection input, Section 3.11), and are re-scored at a looser tolerance to confirm the schedule *responds* to $\gamma_{\min}$ (feasibility and terminal behavior must change); a schedule invariant to the tolerance is rejected regardless of AEP. The held-out AEP never leaves this stage: only feasibility booleans and a margin-over-floor boolean cross the firewall (Section 3.10).

3.8 Fitness and feasibility

For a training cell c, over the shared paired seed set, the score is the percentage improvement in mean AEP over a reference schedule,

$$\text{score}_c \;=\; 100 \times \frac{\overline{\text{AEP}}_{\text{cand},c} - \overline{\text{AEP}}_{\text{ref},c}}{\overline{\text{AEP}}_{\text{ref},c}}, \tag{6}$$

where the reference is the native TopFarm monotone schedule at a rotor-diameter-scaled learning rate, $\eta_{\text{peak}} = c_0 D$ with $c_0 = 0.8333$ (= 200/240) calibrated once on the training farm, decaying to $\gamma_{\min}$. Expressing fitness as percent-over-reference makes cells of very different magnitude (thousands of

GWh for the large offshore farms, hundreds for the small onshore multi-zone farm) commensurable. The reference AEP $\overline{\mathrm{AEP}}_{\mathrm{ref},c}$ is used purely as a *scale constant*: it normalizes the candidate even on a cell where the reference itself is infeasible, and the reference's feasibility never confers credit.

Feasibility at $\gamma_{\min}$ is a *hard gate*: a candidate must be feasible on every paired seed of every training cell, or it is rejected (its score is set to $-\infty$ and it is excluded from the archive as infeasible). The cross-cell aggregate is *farm-balanced*: the mean over farms of each farm's mean cell score, with a worst-cell tiebreak,

$$\text{fitness} \;=\; \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}}\frac{1}{|C_f|}\sum_{c\in C_f}\text{score}_c, \qquad \text{tiebreak} \;=\; \min_{c}\text{score}_c, \tag{7}$$

where $\mathcal{F}$ is the set of farms and C_f the scored cells on farm f. Averaging over farms rather than over cells makes each farm contribute equally irrespective of how many cells it carries (the training set has more cells on the large offshore farm than on the small multi-zone farm), so a percent improvement is worth the same on either farm; the worst-cell tiebreak still prevents a large win on one easy cell from masking a regression on a hard one. The aggregate is taken over the *scored* cells. A cell whose objective is saturated — for example a unidirectional resource in which every constraint-feasible layout that separates the turbines reaches free-stream power, so all such layouts score identically — carries no AEP-improvement signal; it is retained as a *feasibility-only* gate (evaluated at two seeds, participating in the hard feasibility gate) but is excluded from the scored set.

3.9 Provenance and lineage

Every candidate is recorded in an append-only lineage log with a content hash, its parent identifier(s), the mutation engine and model string, the token count and monetary cost, its behavioral descriptors, its per-cell fitness, and its generation and island. Seeded ancestors are logged as generation 0 together with their port transforms. Because the seeds and their transforms are recorded, any later claim that the system *discovered* a schedule family is measured against what was seeded, and "discovered" is distinguishable from "recombined a seed."

3.10 Firewall and scoped workspace

The search must not overfit to, or leak, the held-out and test farms. Two disciplines enforce this. First, *information firewall*: only firewall-safe feedback — per-cell percentage scores and feasibility booleans — ever reaches a mutation engine; held-out and test AEP values never enter a mutator's context or transcript, and the pre-registration of the selection and test design (Section 3.11) lives outside the search entirely. Second, *workspace scoping*: before each mutation the engine is run inside a freshly materialized clean-room directory that contains only the schedule interface, a sanitized copy of the skeleton and seed schedules, and the firewall-safe feedback; the results, analysis, held-out state, and pre-registration are outside the engine's working directory and tool scope. Sanitization strips provenance and redacts any residual path token from the reference code, and a gate check refuses to launch if a forbidden token or a syntactically invalid reference file is present in the scoped workspace.

3.11 Selection and deployment criterion

Selection is margin-aware and never a single-evaluation argmax. Among the archive elites feasible across the training cells, the deployed schedule is the one that maximizes the farm-balanced *training* fitness (Equation (7)) under the hard feasibility gate and worst-cell tiebreak, with ties and within-floor margins broken by a paired, significance-aware comparison so that a schedule is not deployed on a within-noise win; if no elite improves on native under the gate, the native seed is deployed and the run is recorded as producing no improvement. Crucially, the held-out ROWP farm is *not* used for selection: it is firewalled from the search and reserved as an out-of-sample check of the already-selected schedule

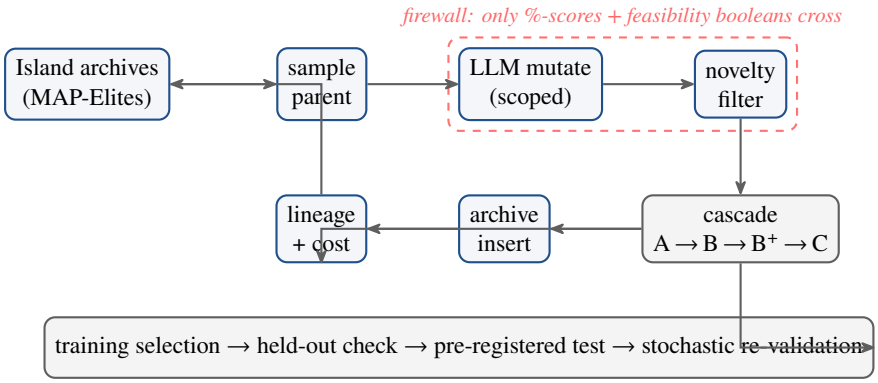

Figure 1: The discovery loop. Parents are sampled from per-island MAP-Elites archives; an LLM mutation engine, run inside a scoped clean-room workspace, proposes a child schedule from the parent source and firewall-safe feedback; a novelty filter rejects near-duplicates before evaluation; the cascade evaluator scores survivors; feasible elites are archived and every candidate is logged with full provenance. Held-out selection, the pre-registered paired test, and stochastic re-validation sit outside the loop.

(Section 6), so the held-out best-of-K result is a genuine generalization test rather than the quantity that was maximized. The deployment claim is *then to be* confirmed, once, on a pre-registered test set under a paired multi-seed protocol (identical per-seed initial layouts across arms) and, as a fidelity stage, re-validated in the full stochastic optimizer; the present paper reports the training-selected schedule and its held-out best-of-K deployment outcome (Section 6) and treats the pre-registered test as the confirmatory step (Section 8). Because feasibility is judged against a tolerance and reference results depend on it, all comparisons are reported at matched tolerance.

4 Application

We apply the framework across three wind-farm geometries and four wind resources. The farms span a range of rotor diameters and constraint geometries: the Danish Energy Island (DEI) cluster with IEA 15 MW turbines ($D = 240$ m, single polygon); the IEA Wind 740-10 Reference Offshore Wind Plant (ROWP), with IEA 10 MW turbines ($D = 198$ m, single polygon), held out from the search as an out-of-sample check; and the Parque Ficticio inclusion-zone case of Criado Risco et al. (2024) with Vestas V80 turbines ($D = 80$ m, five disconnected inclusion zones — the hardest feasibility geometry). Minimum spacing is four rotor diameters for DEI and ROWP and two rotor diameters for Parque. We consider four wind resources: the measured DEI and ROWP roses, an omnidirectional resource, and a unidirectional resource as limiting cases. Figure 2 gives an overview of the geometries and resources and of the train/held-out/test roles.

Training cells. The search optimizes and sees percentage feedback on the training cells of Table 2, which span two rotor diameters, turbine counts from 10 to 120, three resource types (the DEI rose, omnidirectional, and unidirectional — the ROWP rose is held out), the multi-zone geometry, and a high-count cell. Two cheap cells serve as the Stage-A fast-reject filter. The reference AEP column is the scale constant of Equation (6), and the floor column is the per-cell texture floor. One Parque cell under the unidirectional resource is a feasibility-only gate: its objective is saturated (Section 3.8), so it is scored for feasibility only and excluded from the aggregate. The two capability-frontier cells (Section 3.7) — a high-count DEI cell and a dense Parque cell under the unidirectional resource, on which even the reference is infeasible — are not training cells; they are elite-tier probes only. In the runs reported here no candidate reached strict feasibility on either capability-frontier cell, so we make no capability-frontier claim; they remain open probes.

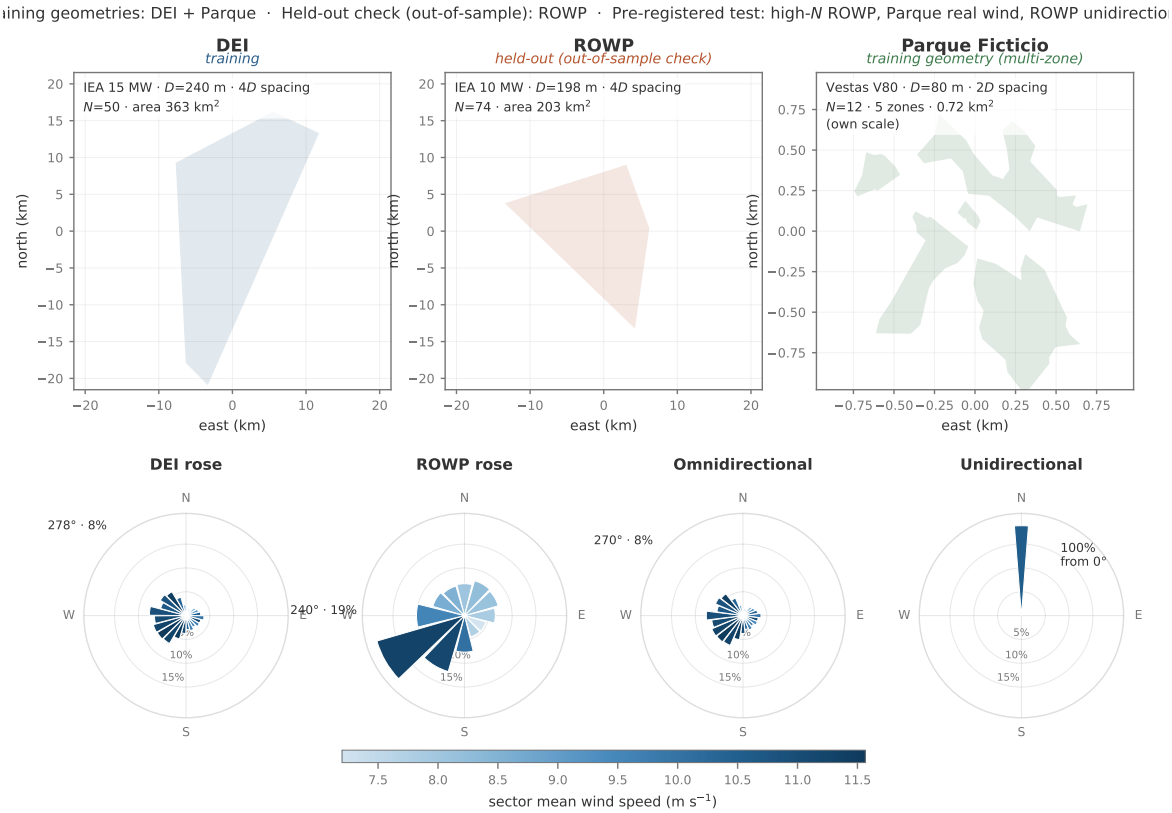

Figure 2: Application overview. *Top:* the three farm geometries — DEI (training, D=240 m) and ROWP (held out as an out-of-sample check, D=198 m) on a shared scale, and the multi-zone Parque Ficticio training geometry (D=80 m, five inclusion zones, shown on its own much smaller scale). *Bottom:* the four wind resources — the measured DEI and ROWP roses, an omnidirectional resource, and a unidirectional limiting case (bar length is the sector energy weight, colour the sector mean speed). Unlike a one-geometry-per-role split, funwake-2 trains on the DEI *and* Parque geometries across turbine counts and roses, selects on the training farms and holds ROWP out as an out-of-sample check, and reserves a pre-registered test set (high-N ROWP, Parque under its real heterogeneous wind, and a ROWP unidirectional extreme).

Held-out and test farms. ROWP is an intermediate scale not present in training and is used only as an out-of-sample check (not a selection input); its AEP is firewalled from the search. The pre-registered test set — to be touched exactly once, at the confirmatory stage (Section 8), and not yet executed in this paper — comprises the high-count ROWP cells (N = 200 and N = 300), the Parque site's real heterogeneous wind resource, and a unidirectional extreme on the held-out ROWP farm.

Evaluation-texture floors. AEP is a mildly discontinuous function of the layout because the wake model masks each turbine's contribution outside a cone; at millimetre-scale perturbations this yields a small texture in re-scored AEP. We measure this floor per farm and set the per-cell minimum meaningful margin accordingly: 0.3 GWh for the large offshore DEI farm, 0.1 GWh for the multi-zone Parque farm, and 0.64 GWh for the held-out ROWP farm. Selection margins use these per-cell floors rather than a single global threshold; the gross Stage-A filter (Section 3.7) does not use them.

4.1 Reference multistart budget and its convergence

The score of Equation (6) compares a candidate against a single reference run per seed, but the baseline a *deployed* schedule must actually beat is stronger: the best feasible layout a practitioner would field

Table 2: Training cells (all references feasible on every seed). D in metres; Ref. AEP is the rotor-diameter-scaled reference (the scale constant of Equation (6)), a multi-seed feasible mean in GWh; Floor is the per-cell evaluation-texture floor (GWh). "feas.-only" marks the saturated-objective cell that gates feasibility but is excluded from the score aggregate.

Cell	Farm	N	D	Resource	Ref. AEP	Floor
DEI-50	DEI	50	240	DEI rose	5560.4	0.3
DEI-80	DEI	80	240	omnidirectional	8818.1	0.3
DEI-120	DEI	120	240	DEI rose	13030.0	0.3
DEI-50 uni.	DEI	50	240	unidirectional	5598.4	0.3
Parque-20	Parque	20	80	DEI rose	231.5	0.1
Parque-14 uni.	Parque	14	80	unidirectional	*184.8 (f.o.)*	0.1
Parque-10	Parque	10	80	omnidirectional	127.0	0.1

from a *multistart* of K independent random initializations of the reference. The number of starts that defines a converged multistart is not arbitrary; we fix it per farm with the shuffle-saturation analysis of Quick et al. (2026). A pool of M feasible reference runs is collected on a farm; the pool is randomly reshuffled 10^3 times, and for each shuffle the running maximum AEP is recorded against the number of starts considered. The p-th percentile of that running maximum across shuffles is a conservative estimate of the best AEP obtained with that many starts — the first percentile is the value beaten 99% of the time — and the convergence budget $K^\star$ is the number of starts at which that percentile reaches within a tolerance of the pool maximum (made precise below). The pool is large enough to resolve $K^\star$ when $K^\star \le M/2$; otherwise the pool is doubled and the analysis repeated.

Because AEP is effectively continuous near the optimum, the single best run of a pool is an isolated point, so reaching the *exact* pool maximum saturates only at $\approx 0.99\,M$ regardless of M. We therefore read saturation at a physically negligible tolerance ϵ (reaching within ϵ of the pool maximum) and report $K^\star$ across ϵ. The budget is then a property of the density of the near-optimum basin: if a fraction f_ϵ of runs land within ϵ of the best, the first-percentile budget approaches $\ln(0.01)/\ln(1-f_\epsilon)$ as the pool grows.

We calibrate on the DEI training farm and the held-out ROWP farm with pools of $M \approx 4000$ feasible reference runs each, collected as HPC array jobs (Figure 3 and Table 3). The per-pool convergence estimate is itself noisy at small pools — on the held-out farm the first-percentile budget is 420 ± 214 at $M = 1024$ against ≈ 1070 at the deep pool — so it is the deep pools with 10^3-shuffle averaging, not a single half-pool check, that make $K^\star$ trustworthy; at the deep pools the half-pool rule holds on both farms ($K^\star_1 \le M/2$), confirming the pool is large enough to resolve the budget. ROWP is consistently the harder farm: its near-optimum basin is sparser (0.39% of runs within 0.05% of the best against 0.52% for DEI, and 3.5% vs. 9.3% within 0.1%), so it requires 1.3–3× more starts than the training farm. At the tenth percentile the budget is ≈ 410 starts for DEI and ≈ 570 for ROWP; for first-percentile confidence within 0.05% of the optimum it is ≈ 780 and ≈ 1070 (asymptotically ≈ 880 and ≈ 1180). The 400-start choice of Quick et al. (2026) for their DEI farm is close to our tenth-percentile DEI budget, consistent with this calibration.

We adopt a reference multistart of $K = 1024$ starts. This exceeds the first-percentile budget on the training farm ($K^\star_1 \approx 784$) and the fifth-percentile budget on the held-out farm ($K^\star_5 \approx 728$), and covers the held-out first percentile at the 0.1% tolerance (≈ 121); at the strictest 0.05% tolerance it sits just below the held-out first-percentile budget (≈ 1070), a shortfall we regard as immaterial at that margin. Deployment comparisons (Section 3.11) are made against the best-of-K of this multistart at matched start count, not against its mean.

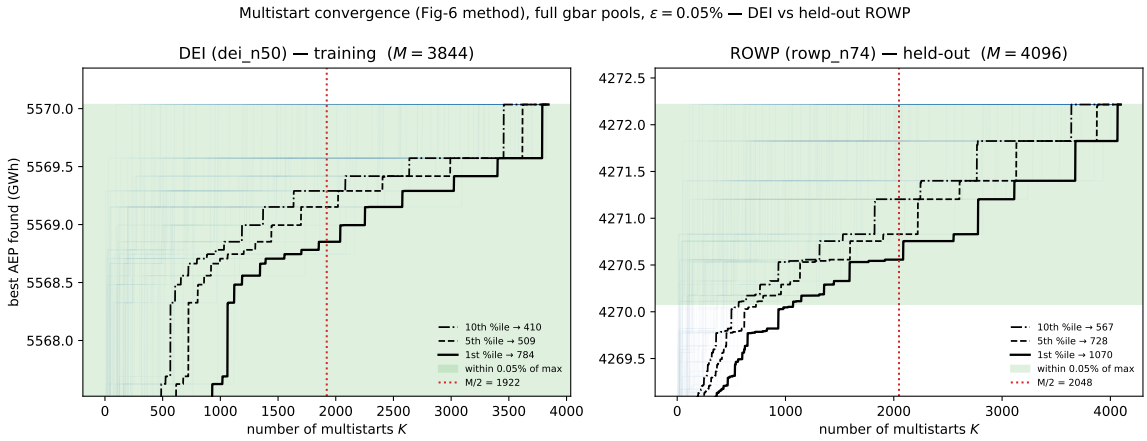

Figure 3: Shuffle-saturation convergence of the reference multistart on the DEI training farm (left) and the held-out ROWP farm (right), following Quick et al. (2026). Faint lines are individual shuffles of the pool; the black curves are the tenth-, fifth-, and first-percentile running maxima; the shaded band is within $\epsilon = 0.05\%$ of the pool maximum, and the dotted vertical line is the half-pool budget $M/2$. Each percentile reaches the band well inside $M/2$ on both farms; the held-out ROWP farm (sparser near-optimum basin) requires more starts than the training farm.

Table 3: Reference-multistart convergence budget per farm, computed on the full deep pools ($M = 3844$ for DEI, $M = 4096$ for ROWP) that back Figure 3. "spread" is the relative AEP range $(\max - \min)/\max$ over the pool; f_ϵ is the fraction of runs within ϵ of the pool maximum; $K_p^\star$ is the number of starts at which the p-th-percentile running maximum reaches within ϵ of the pool maximum (at $\epsilon = 0.05\%$ for the 10th/5th/1st columns, and 0.1% for the last). The held-out ROWP farm needs 1.3–3× more starts than the DEI training farm, and both satisfy $K_1^\star \le M/2$.

Farm	spread (%)	$f_{0.05\%}$ (%)	$f_{0.1\%}$ (%)	$K_{10}^\star$ (0.05%)	$K_5^\star$ (0.05%)	$K_1^\star$ (0.05%)	$K_1^\star$ (0.1%)
DEI (M=3844)	0.282	0.52	9.3	410	509	784	47
ROWP (M=4096)	0.456	0.39	3.5	567	728	1070	121

5 Discovered schedules

Figure 4 shows a schedule search on the training farm for two mutation engines — 557 scored attempts for the Claude engine and 867 for the Antigravity/Gemini-3 engine. Each point is one candidate the LLM proposed and ran through the fixed skeleton; filled points are feasible on every seed, hollow points are infeasible, and the solid line is the best feasible fitness so far. Most candidates land within a few hundredths of a percent of the native reference, the best feasible schedule reaches +0.068% (Claude) and +0.079% (Antigravity), the best-so-far improves by only ≈0.05–0.09% over hundreds of attempts, and the infeasible outliers near +0.7% in the Claude panel are exactly the non-deployable layouts the feasibility gate (Section 3.8) discards — a reminder that raw AEP off the feasible set is not a fitness signal (the Antigravity engine's largest infeasible score is only +0.05%). The two searches shown are single-farm engine runs that illustrate the search dynamics and the diversity of shapes; the deployed schedule it21 comes from a separate farm-balanced portfolio search of ∼120 attempts over a five-cell subset of the training suite (both DEI single-polygon roses, the DEI omnidirectional and unidirectional cells, and the two Parque cells; the high-count DEI-120 cell is an elite-tier probe, not part of this portfolio) — the elite that maximizes the farm-balanced training fitness of Equation (7) (Section 3.11). The picture is a faithful near-null: the interface admits broad exploration, but the scale-aware native reference is already close to the best a schedule achieves on these cells.

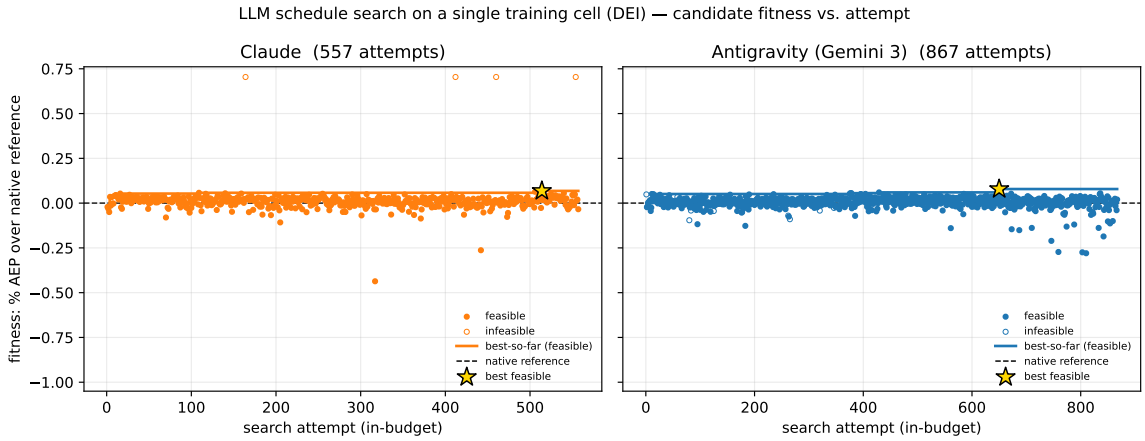

Figure 4: Schedule search on a single training cell (DEI) for two of the mutation engines (Claude, left; Antigravity/Gemini 3, right). Each point is a candidate schedule proposed by the LLM and scored by the fixed skeleton; filled points are feasible on every seed, hollow points are infeasible; the solid line is the best feasible fitness so far; the dashed line is the native reference (0%); the star marks the best feasible schedule found by that engine. The deployed schedule it21 is *not* among these points — it comes from a separate farm-balanced codex portfolio run (Section 3.11) — so these panels illustrate search dynamics, not the deployment decision. The best-so-far improves only $\approx$0.05–0.09% over the native reference, and the infeasible +0.7% outliers (in the Claude panel) are non-deployable layouts discarded by the feasibility gate.

Given the near-null AEP, the more useful observation is the *range of deployable schedule shapes the interface admits*. Figure 5 plots four schedules evaluated at the held-out ROWP scales: the native reference (constant learning rate then a compounding-product decay, penalty coupled as $\alpha \propto 1/\eta$, fixed TopFarm moment coefficients β_1=0.1, β_2=0.2); the one new feasible shape the portfolio search itself surfaced (it21), which ramps quickly to the exploration scale, holds, then decays smoothly while *raising* the adaptive coefficient β_2 from 0.2 to 0.77 and easing the momentum β_1 down into the tail; and two *seeded* ancestors — a dual-bump learning rate (it192) and a cyclic warm-restart schedule (it118) — that were carried in from earlier single-farm runs and mark the extremes the interface admits. We are explicit that the two extreme shapes are seeds, not discoveries: the search's own contribution here is the single feasible it21 family; the value of the exercise is that all these distinct, hand-inscrutable shapes decay the learning rate toward $\gamma_{\min}$, restore the penalty terminally, and attain comparable feasible AEP — the objective is forgiving in schedule space once the scale-aware exploration and terminal feasibility restoration are right.

6 Deployment and generalization

Deployment fields the *best feasible layout* a schedule produces over a K-start multistart, not its mean (Section 3.11), so we compare the selected schedule against the native reference at the calibrated budget $K = 1024$ (Section 4.1) on the held-out ROWP farm. Figure 6 bootstraps the best-of-K AEP of both schedules from independent deep pools (4096 native runs, all feasible; 2048 runs of the portfolio-discovered schedule it21, 99.7% feasible). The discovered schedule's advantage is stable across the whole budget range; at $K = 1024$ its best-of-K layout beats the native best-of-K by +3.2 GWh — +0.075% of the native best, with a bootstrap 95% confidence interval of [+1.3, +4.9] GWh that excludes zero (the discovered schedule wins in every one of 4000 bootstrap replicates). The advantage is a distributional shift, not a lucky tail: the discovered schedule's ROWP pool mean exceeds native's by +0.065% and its pool maximum by +3.0 GWh. Because ROWP was firewalled from the search, this is an honest out-of-sample generalization of a schedule selected only on the training farms.

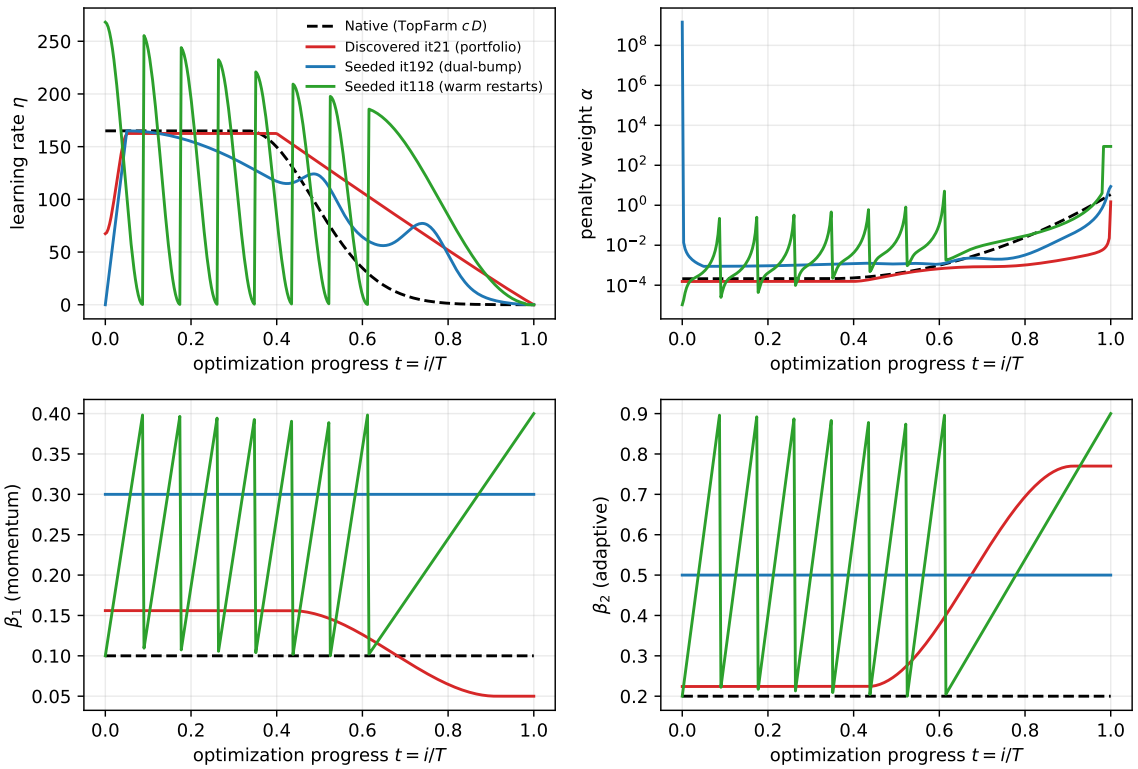

Figure 5: Four schedules evaluated at the held-out ROWP scales (D = 198 m, $D_{\min}$ = 792 m, N = 74, $\gamma_{\min}$ = 0.01, and α_0 from the skeleton): learning rate η, penalty weight α (log scale), and the two Adam moment coefficients β_1, β_2 over optimization progress $t = i/T$. The native reference (dashed) is shown against a portfolio-discovered schedule (it21) and two seeded ancestors (dual-bump it192; cyclic warm-restart it118). The interface admits widely different shapes; all decay η toward $\gamma_{\min}$ and restore the penalty terminally.

This held-out win does *not* generalize uniformly, and the cross-farm picture is noisy. Table 4 re-scores the discovered schedule across the training matrix: the DEI and ROWP single-polygon cells improve by small, consistent margins (roughly +0.015 to +0.055%), while the two multi-zone Parque cells are high-variance and *mixed* — a −0.10% loss on the 20-turbine cell and a +0.35% gain on the 10-turbine cell. At four paired seeds per cell these Parque figures are within noise: the ROWP paired-difference standard deviation alone is ≈ 0.09%, so a 0.05% cell effect needs of order 27 seeds to resolve at 80% power. We therefore read the table for its qualitative message — the discovered schedule is a small, *uneven* improver over native rather than a uniform replacement, and the farm-balanced mean (+0.07%) is small and dominated by the noisy Parque terms — and treat the deeply-sampled held-out ROWP best-of-K result above as the one clean signal. The search itself is near-null in mean terms (Figure 4); the deployable value comes from selecting the best of a converged multistart, where the discovered schedule's distributional edge compounds.

Two caveats temper the result. The discovered schedule is the argmax of a finite search, a selection that the held-out evaluation mitigates but does not remove; and it is marginally less robust on feasibility than the native reference (99.7% vs. 100% of ROWP starts feasible). The hard feasibility gate (Section 3.8) is load-bearing here: infeasible layouts can post large *apparent* AEP gains (they place turbines outside the buildable region or below the spacing bound), so a fitness that credited raw AEP off the feasible set would reward non-deployable candidates — the hollow outliers of Figure 4 — and select them for deployment. Every comparison in this section is between feasible-only best-of-K layouts at matched tolerance.

Table 4: Generalization of the discovered schedule across the five scored portfolio cells plus the held-out ROWP cell (percentage change in mean AEP over the native reference, four paired seeds per cell, both schedules feasible on every seed; the high-count DEI-120 cell of Table 2 is an elite-tier probe and is not re-scored here). Gains concentrate on the offshore single-polygon cells; the dense onshore multi-zone Parque cells are neutral-to-negative. Per-cell estimates are noisy (see text); the held-out ROWP row is the clean out-of-sample cell.

Cell	Family	Δ mean AEP
DEI-50 (rose)	offshore single-poly	+0.015%
DEI-80 (omnidirectional)	offshore single-poly	+0.054%
DEI-50 (unidirectional)	offshore single-poly	+0.017%
Parque-10 (omnidir.)	onshore multi-zone	+0.352%
Parque-20 (rose)	onshore multi-zone	−0.099%
ROWP-74 (held out)	offshore single-poly	+0.051%

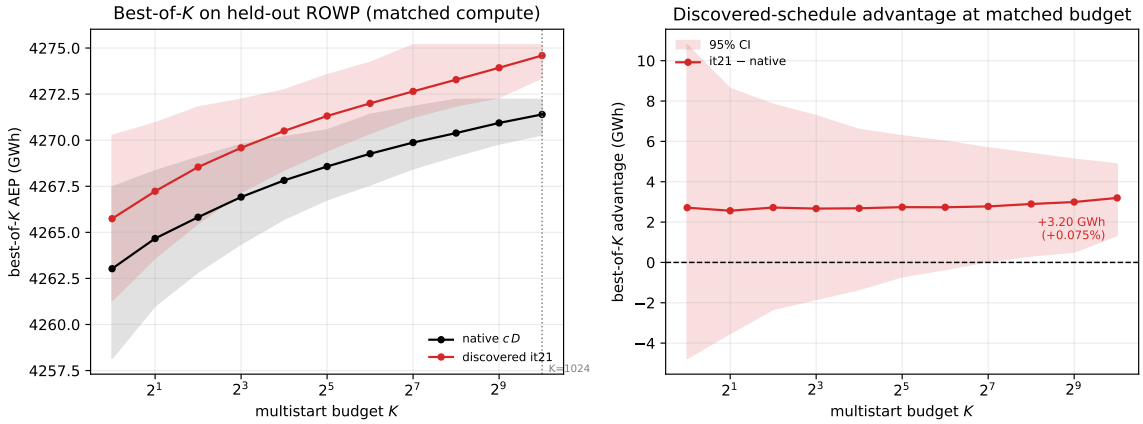

Figure 6: Best-of-K deployment comparison on the held-out ROWP farm at matched compute, bootstrapped from independent deep pools (native 4096 runs; discovered it21 2048 runs). *Left:* best-of-K AEP versus multistart budget K (median with 5–95th percentile band); the discovered schedule sits above the native reference at every budget. *Right:* the best-of-K advantage (discovered − native) with a 95% bootstrap band; at the calibrated budget $K = 1024$ it is +3.2 GWh (+0.075%) and the interval excludes zero.

7 Reproducibility and cost control

The framework is engineered to be re-runnable and auditable. A content-addressed cache keyed by (schedule hash, cell, seed, $\gamma_{\min}, T$) returns a stored result for any previously computed evaluation, so a resumed run recomputes nothing. State — the archives, the novelty seen-set, the cost tally, and the generation counter — is checkpointed atomically each generation; the parent-sampling randomness is derived from the run identifier and generation, and parent lists are ordered canonically, so a resumed run reproduces the same candidates and hits the cache for every evaluation. The single-precision canonicalization of α_0 (Section 3.4) makes each evaluation *bit-identical across independent processes on a fixed platform*. Across different hardware architectures a small floating-point drift remains — of order ±1.7 GWh on the large offshore farm between arm64 and x86 in our measurements — so cross-platform results are compared at that measured drift tolerance rather than as bit-identity. A cost ceiling tracks cumulative tokens and spend from the per-invocation engine logs and cleanly aborts the run near the ceiling (it stops issuing new mutations, finishes in-flight work, checkpoints, and stops). Together these make a discovery run a deterministic function of its seeds, configuration, and budget on

a fixed platform.

8 Limitations

The effects are small and the evidence is deliberately conservative. *(i) Magnitude and selection.* The best-so-far search improves only ≈ 0.05–0.09% over the native reference (Figure 4), and the deployed schedule is the argmax of a finite (farm-balanced portfolio search, ~ 120 attempts) search; the held-out evaluation and the paired, significance-aware selection mitigate but do not remove this selection bias. *(ii) Farm specificity.* The held-out best-of-K win is real, but across the training matrix the discovered schedule is a small and *uneven* improver (Table 4), not a uniform replacement for the native reference; its per-cell effects on the multi-zone Parque farm are mixed and, at four seeds, within noise. *(iii) Feasibility as fitness.* Feasibility is load-bearing: an infeasible layout can post a large *phantom* AEP gain — we observed a +14.5% "improvement" from an infeasible 20-turbine Parque layout that places turbines outside the buildable region — so a fitness that credited AEP off the feasible set, or a deployment metric that averaged it, would select non-deployable candidates. An earlier version of our own deployment metric did exactly this and produced a spurious "generalist beats native" result that we retracted; the hard feasibility gate (Section 3.8) and the feasibility-penalized, significance-aware selection reported here are the fix. *(iv) Scope of the reported evaluation.* The pre-registered test set (Section 4) and the full-stochastic re-validation are defined but not yet executed here; we report the held-out farm, and treat the pre-registered test as the confirmatory step for future work. *(v) Fidelity.* Results are conditional on one engineering wake model, one skeleton, and a finite seed budget; per-cell generalization figures in particular are four-seed estimates and individually noisy. The reported optimizations use our reimplemented gradient skeleton (which computes real wake-model AEP, not a surrogate); agreement with TopFarm's production SGD driver is the fidelity step (Section 3.11) and is not yet reported. *(vi) Cost reporting.* We describe the cost-control mechanism but do not tabulate absolute token, dollar, or wall-clock figures for the runs in this paper; per-invocation totals are logged but not aggregated here.

9 Conclusions

We described a framework for discovering deployable wind-farm layout optimizers with large language models. Its distinguishing features are a scale-aware schedule interface that removes the free learning rate and builds the exploration scale from the rotor diameter; a quality-diversity evolutionary loop whose cascade evaluator scores candidates across a multi-farm, multi-scale training suite under a hard feasibility gate; and a provenance, firewall, and reproducibility layer — lineage, a behavioral archive, content-addressed caching, a scoped mutator workspace, and a pre-registered selection and deployment criterion — that make the procedure auditable and repeatable.

Empirically, the search is near-null in mean AEP: discovered schedules improve on the native scale-aware reference by only ≈ 0.05–0.09%, and the diversity of feasible schedule shapes the interface admits exceeds the marginal objective gain. The deployable value appears at the metric a practitioner actually fields — the best of a converged multistart — where a discovered schedule beats native by $+0.075\%$ on a held-out farm firewalled from the search, with a bootstrap interval excluding zero. That gain is small, farm-specific, and obtained from a search argmax, so the honest output of the procedure is a *candidate* schedule together with a pre-registered protocol to confirm or refute it on unseen farms — which is what an engineering discovery procedure, as opposed to a one-off tuning, should deliver. Whether LLM-discovered schedules can beat a well-calibrated scale-aware reference by margins that matter at fleet scale remains open; the contribution here is the repeatable, auditable, firewalled machinery to ask that question and to keep its answers honest.

A Schedule interface and mutation prompt

The searched object is a single Python function; the skeleton, seeds, and firewall-safe feedback are provided read-only in the scoped workspace.

```
def schedule_fn(step, total_steps, D, min_spacing,
                n_turbines, gamma_min, alpha0):
    """Return (lr, alpha, beta1, beta2) at `step`.

    D          : rotor diameter (m)            -- exploration-scale source
    min_spacing: minimum inter-turbine spacing (m)
    n_turbines : N                             -- density / packing signal
    gamma_min  : constraint tolerance (m)      -- the terminal learning rate,
                                                  the only user-supplied scale
    alpha0     : penalty normalizer = mean(|grad J|)/D  (skeleton-computed)
    """
    ...
    return lr, alpha, beta1, beta2
```

The mutation engine receives, in its scoped workspace: the interface above, a sanitized copy of the fixed skeleton and the seed schedules, the parent schedule source, and the firewall-safe feedback (per-cell percentage scores and feasibility booleans). It returns one improved module defining `schedule_fn`. No held-out or test AEP, and none of the analysis or pre-registration, is ever placed in that workspace.

References

Bastankhah, M. and Porté-Agel, F.: A new analytical model for wind-turbine wakes, *Renewable Energy*, 70, 116–123, 2014.

Criado Risco, J., Valotta Rodrigues, R., Friis-Møller, M., Quick, J., Mølgaard Pedersen, M., and Réthoré, P.-E.: Gradient-based wind farm layout optimization with inclusion and exclusion zones, *Wind Energy Science*, 9, 585–600, 2024.

Google DeepMind: AlphaEvolve: a coding agent for scientific and algorithmic discovery, Technical report, 2025.

Kingma, D. P. and Ba, J.: Adam: a method for stochastic optimization, in *International Conference on Learning Representations*, 2015.

Mouret, J.-B. and Clune, J.: Illuminating search spaces by mapping elites, arXiv preprint arXiv:1504.04909, 2015.

OpenEvolve: an open-source evolutionary coding framework, software, 2025.

Quick, J., Réthoré, P.-E., Mølgaard Pedersen, M., and Rodrigues, R. V.: Stochastic gradient descent for wind farm optimization, *Wind Energy Science Discussions*, 1–17, 2022.

Quick, J., Réthoré, P.-E., Maak, L., and Maheshwari, P.: Wind farm design under uncertainty: managing unknown neighboring farm effects, in *Proceedings of the ASME 2026 45th International Conference on Ocean, Offshore and Arctic Engineering (OMAE)*, OMAE2026-176207, 2026.

Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J., Ellenberg, J. S., Wang, P., Fawzi, O., et al.: Mathematical discoveries from program search with large language models, *Nature*, 625, 468–475, 2024.

Sakana AI: ShinkaEvolve: sample-efficient evolutionary program search with large language models, software/technical report, 2025.

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., and Chen, X.: Large language models as optimizers, arXiv preprint arXiv:2309.03409, 2023.

Ma, Y. J., Liang, W., Wang, G., Huang, D.-A., Bastani, O., Jayaraman, D., Zhu, Y., Fan, L., and Anandkumar, A.: Eureka: human-level reward design via coding large language models, arXiv preprint arXiv:2310.12931, 2023.